



Python Regex

Cheatsheet

Learn to read patterns once — then reuse them
everywhere in your code.



Chandra Sekhar

AI Engineer & Trainer

Three Core re Entry Points

RE.MATCH()

Succeeds only when the pattern lines up with the **first character** — nothing before it is considered.

```
Python  
import re  
re.match(r'Hi\b', 'Hi there, friend')
```

RE.SEARCH()

Walks left to right and returns the **first** substring that fits — offset does not matter.

```
Python  
re.search(r'there', 'Oh, there you are')
```

RE.FINDALL()

Collects every separate hit into a list, skipping overlaps by default.

```
Python  
re.findall(r'\d+', 'Batch-42 has 7 items and 3 spares')
```



Anchors & Repetition

`.` = one code unit except line break · `^` = string start · `$` = string end (unless flags say otherwise).

 Python

```
re.search(r'^BEGIN.*DONE$', 'BEGIN middle bit DONE')
```

REPEAT OPERATORS

`*` repeat 0.. ∞ · `+` repeat 1.. ∞ · `?` optional (0 or 1).

 Python

```
re.findall(r'xy*', 'xyyyy') # whole span 'xyyyy'
```

Remember: Pair anchors with quantifiers when you need tight control over how much text each piece may consume.



Built-in Character Shorthands

NUMERIC VS NON-NUMERIC

`\d` matches Unicode digits · `\D` is the inverse set.

 Python

```
re.findall(r'\d', 'SKU #8840-A')  
re.findall(r'\D', 'SKU #8840-A')
```

IDENTIFIER-FRIENDLY RUNS

`\w` letters, digits, underscore · `\W` punctuation and symbols between tokens.

 Python

```
re.findall(r'\w+', 'user_2@host! ok')
```

SPACE VS TOKENS

`\s` tabs, spaces, common line breaks · `\S` “visible” chunks.

 Python

```
re.findall(r'\s', 'x y z')  
re.findall(r'\S+', 'x y z')
```



Custom Character Classes

Square brackets mean “pick **one** member.” Ranges like `[a-z]` expand inline; a leading `^` flips the meaning.

 Python

```
re.findall(r'[aeiou]', 'algorithms')
```

 Python

```
re.findall(r'^[0-9]', 'Unit-55B') # hyphens, letters...
```

Think of each bracket expression as a miniature alphabet you allow for a single slot.

Parentheses & Alternation

`(...)` stores sub-matches you can read with `.group(n)` · `|` chooses branches · `(?:...)` groups without saving.

 Python

```
m = re.search(r'(\d\d):(\d\d)', 'Event at 14:05 UTC')
print(m.group(2)) # minutes
```

 Python

```
re.findall(r'tea|coffee', 'Prefer tea over coffee today')
```

Transform & Reuse Patterns

RE.SUB()

Swap each hit for a replacement (plain text or a function for dynamic output).

 Python

```
re.sub(r'ERROR', 'WARN', 'ERROR: disk full - ERROR: retry')
```

RE.SPLIT()

Cut the string wherever the delimiter regex matches; delimiters themselves disappear from the list.

 Python

```
re.split(r'\s*;\s*', 'alpha; beta ;gamma')
```

RE.COMPILE()

Parse the pattern once, then call `.search` / `.sub` repeatedly without re-parsing overhead.

 Python

```
rx = re.compile(r'\d\d\d')  
rx.findall('codes 009 and 441')
```



Optional Match Modes

re.IGNORECASE (alias **re.I**) ignores letter casing when comparing.

 Python

```
re.findall(r'regex', 'ReGeX is fun', re.IGNORECASE)
```

re.MULTILINE — treat **^** / **\$** as line boundaries inside multiline text.

 Python

```
re.findall(r'^\w+', "First\nSecond line", re.MULTILINE)
```

re.DOTALL — lets **.** swallow newline characters too.

 Python

```
re.search(r'.+', 'line1\nline2', re.DOTALL)
```



Greedy vs Lazy Quantifiers

Out of the box, ***** and **+** grab the **longest** slice that still lets the rest of the pattern succeed.



Python

```
re.search(r'".*"', 'say "hi" then "bye"').group()  
# greedy: spans both quoted phrases
```

Tack on **?** after the quantifier for **shortest** valid span (often what you want inside markup or logs).



Python

```
re.search(r'".*?"', 'say "hi" then "bye"').group()  
# non-greedy: stops at first closing quote
```

When extraction feels "too wide," try ***?** / **+?** before rewriting the whole regex.





Chandra Sekhar

AI Engineer and Trainer



Follow for daily AI interview
questions, tips & career advice.

● [Follow on LinkedIn →](#)